Reflection and Traceability Report on Software Engineering

Team 8, AlgoCatan
Jake Read
Rebecca Di Filippo
Matthew Cheung
Sunny Yao

[Reflection is an important component of getting the full benefits from a learning experience. Besides the intrinsic benefits of reflection, this document will be used to help the TAs grade how well your team responded to feedback. Therefore, traceability between Revision 0 and Revision 1 is and important part of the reflection exercise. In addition, several CEAB (Canadian Engineering Accreditation Board) Learning Outcomes (LOs) will be assessed based on your reflections. —TPLT]

1 Changes in Response to Feedback

[Summarize the changes made over the course of the project in response to feedback from TAs, the instructor, teammates, other teams, the project supervisor (if present), and from user testers. —TPLT]

[For those teams with an external supervisor, please highlight how the feedback from the supervisor shaped your project. In particular, you should highlight the supervisor's response to your Rev 0 demonstration to them. —TPLT]

[Version control can make the summary relatively easy, if you used issues and meaningful commits. If you feedback is in an issue, and you responded in the issue tracker, you can point to the issue as part of explaining your changes. If addressing the issue required changes to code or documentation, you can point to the specific commit that made the changes. Although the links are helpful for the details, you should include a label for each item of feedback so that the reader has an idea of what each item is about without the need to click on everything to find out. —TPLT]

[If you were not organized with your commits, traceability between feedback and commits will not be feasible to capture after the fact. You will instead need to spend time writing down a summary of the changes made in response to each item of feedback. —TPLT]

[You should address EVERY item of feedback. A table or itemized list is

recommended. You should record every item of feedback, along with the source of that feedback and the change you made in response to that feedback. The response can be a change to your documentation, code, or development process. The response can also be the reason why no changes were made in response to the feedback. To make this information manageable, you will record the feedback and response separately for each deliverable in the sections that follow. —TPLT]

[If the feedback is general or incomplete, the TA (or instructor) will not be able to grade your response to feedback. In that case your grade on this document, and likely the Revision 1 versions of the other documents will be low. —TPLT]

1.1 SRS and Hazard Analysis

Table 1: SRS and HA Feedback and Resultant Changes

Source	Feedback Summary	Link to Issue	Link to Commit
Peer Review	NFR-E.10 and 11 missing values	Issue #16 Issue #17	Commit 983904c
Peer Review	Some stakeholders not defined	Issue #13	Commit d21fdf5
Peer Review	Section S.3 missing important APIs/interfaces	Issue #19	Commit bd9db53
Peer Review	Invariants appear to be duplicated	Issue #18	Commit 4574535
Peer Review	Not all high-level usage scenarios given later details	Issue #14	Commit 849db38
Peer Review	Detailed Usage Scenario poorly detailed and justification from dev rather than user perspective	Issue #15	Commit 849db38
Peer Review	Too few details in high-level usage scenarios	Issue #20	N/A (See issue)
Peer Review	Unclear invariant source	Issue #21	Commit 77bdeac
Peer Review	Add NFRs specific to each component	Issue #23	N/A (See issue)
Peer Review	HA missing legal move assumption	Issue #24	N/A (See issue)
Peer Review	HA camera safety req has too few implementation details	Issue #26	N/A (See issue)
Peer Review	Missing potential loss incurred by system failure	Issue #27	Commit c1f5ec8
Peer Review	FR.Sa.11 not verifiable	Issue #28	Commit 5258f02
Peer Review	FMEA table missing fields	Issue #29	N/A (See issue)
TA Feedback	The wording of the requirements was very poor, and did not follow the typical pattern of "The system shall...". (This was one of the major areas we lost grades on the SRS doc)	Issue #109	Commit c3c3f02
Peer Review	Section S.1 needed clearer component interactions and links between components	Issue #22	Commit 57547fe

Continued on next page

Source	Feedback Summary	Link to Issue	Link to Commit
TA Feedback	Various issues with requirements, including non-verifiable requirements, and some that weren't black-boxes (we only received a level 3 in these areas)	Issue #110	Commit 8cd57fb
Internal Team Feedback	A lot has changed since the SRS was written, and a lot of old requirements are no longer relevant, so we need to go through and remove or update them and their corresponding modules (This change required a full SRS overhaul)	Issue #108	Commit e491f80
Peer Review + Ta Feedback	The inital component diagram was quite poor, and didn't do a good job of showing the interactions between components, so we redesigned it to be more clear and informative	Issue #111 Issue #22	Commit fdf7f07

1.2 Design and Design Documentation

Table 2: Design and Design Documentation Feedback and Resultant Changes

Source	Feedback Summary	Link to Issue	Link to Commit
Peer Review	Module Guide section 7 needed clearer "Implemented By" technology details	Issue #48	Commit 42ad9fb
Peer Review	MIS M8 resetQueue() should not be local-only; promoted to exported access routine	Issue #49	Commit 769daf0
Peer Review	MIS integer constants/state variables (e.g., max turns, score) needed tighter numeric domains	Issue #51	Commit 64a9e40
Peer Review	Module Guide UC.1 needed clearer mapping to stable software decision/rules dependency	Issue #52	Commit 0b0977e
Peer Review	Module Guide section 9 use hierarchy diagram should clarify module dependencies and acknowledge cycles from decoupled architecture	Issue #53	Commit 4a4f580
Peer Review	Module Guide section 6 Clearer Connection between Requirements and Design	Issue #54	Commit 4aa70a0

1.3 VnV Plan and Report

Table 3: VnV Plan and Report Feedback and Resultant Changes

Source	Feedback Summary	Link to Issue	Link to Commit
Peer Review	Potential use of Postman for API testing	Issue #30	N/A (See issue)
Peer Review	SRS peer review plan needs more details	Issue #31	Commit 05dd5dd
Peer Review	Duplicate heading in VnV plan	Issue #32	Commit 2a4d0c
Peer Review	Section 3.7 needed more explicit validation metrics for supervisor demo and usability survey	Issue #34	Commit d928005
Peer Review	Functional requirement traceability table was unclear relative to the non-functional requirements section	Issue #35	Commit 069c83e
Peer Review	VnV Report section 7 needed clearer priority/scope categorization for testing-driven changes	Issue #81	Commit 20306b9
Peer Review	VnV Report and Plan needed more specificity on bash scripts used and rationale for unit testing framework choices	Issue #82	Commit 3fe0c62
Peer Review	VnV Report usability section needed clearer explanation of test categories and expansion on Catan-to-Bot intuition	Issue #83	Commit 9d6e0be
Peer Review	VnV Report Performance test needed specific timing metrics to validate performance requirements	Issue #84	Commit 3b447b1
Peer Review	VnV Report NFR evaluation sections needed more detailed testing methodology and process descriptions	Issue #85	Commit e7d079b
Peer Review	VnV Report Data Integrity tests needed additional unit/component tests for invalid state and action validation	Issue #86	Commit 2ac373a
Peer Review	VnV Plan Performance and Scalability section needed specific performance thresholds and separate test cases	Issue #33	Commit 47af5ac

2 Extras

[Summarize the extras that were tackled by this project. Extras can include usability testing, code walkthroughs, user documentation, formal proof, GenderMag personas, Design Thinking, etc. Extras should have already been approved by the course instructor as included in your problem statement. —TPLT]

For our project, we chose to conduct usability testing, and create a user instructional video for our extras.

The usability testing was conducted in multiple stages throughout the process. We ran the first interviews and subsequent survey following our rev0 demonstration, and the feedback we received heavily guided our design decisions between rev0 and rev1. As such, we will expand further on the details of this feedback in the following design iteration section. Following the incorporation of feedback from this first round of usability testing, we ran a second round of interviews and surveys to evaluate the changes we made in response to the first round of feedback. We saw a noticeable increase in the UX metrics we were tracking, indicating our changes were valuable. For more information on the specifics of this process, see the usability testing document in the extras folder.

The user instructional video was created following the completion of our rev1 documentation and is intended to be a resource for users to learn how to use our product. It's around 3 minutes long, and gives explanations of the various settings and features present in our front-end interface, as well as a demonstration of how to use the product to play a game of *Catan* against the bot. The video is available on our YouTube channel, and the link to the video is included in the extras folder. The responses from users we showed it to are also listed in a doc in the same folder.

3 Design Iteration (LO11 (PrototypeIterate))

[Explain how you arrived at your final design and implementation. How did the design evolve from the first version to the final version? —TPLT]

[Don't just say what you changed, say why you changed it. The needs of the client should be part of the explanation. For example, if you made changes in response to usability testing, explain what the testing found and what changes it led to. —TPLT]

Over the course of the project, our design evolved significantly in response to feedback from various sources, including peer reviews, TA and supervisor feedback, and usability testing. At the beginning of the project, our planned flow was to use computer vision to read the game state off a physical board, and then have a trained reinforcement learning model provide move recommendations through an interface, acting as a sort of coach for the user playing the game. Obviously, our design now has evolved significantly from this initial concept.

The first major change in our plans came quite early in the process. We began by searching for existing *Catan* simulators, as designing a full sim for

an RL model to train on would have been a significant undertaking. We came across Catanatron, an open-source *Catan* simulator written in Python, and decided to use it as the core engine of our project. Having made this decision, we began work on integrating with Catanatron, taking the time to understand the codebase and design of the simulator, as well as its existing bots. We then spent some time designing and implementing some simple bots to test our integration, as well as incorporating Catanatron unit tests into our CI pipeline to ensure our integration didn't break any existing functionality. This decision to use Catanatron was a major turning point for our project, as it meant we could get straight to work on our RL model. We progressed with this model quite quickly early on, and by the time of our rev0 demonstration, we had a working model that could play a game of *Catan* against the existing bots in Catanatron, and beat a random bot consistently. This was a significant milestone, and we decided as a team to defer the development of the computer vision component indefinitely, in order to put more effort into the bot itself. This meant we would no longer be reading off a physical board, and we moved away from the idea of a coach to a more traditional bot that could be played against through Catanatron's CLI.

At this stage we put our full effort into improving our bot, implementing a variety of RL techniques to improve training, such as reward shaping, curriculum learning, and self-play. The bot was progressing steadily, and we were happy with the results we were seeing, but in a meeting with our supervisor Prof. Istvan David, he expressed concern that while our work was impressive, the only method we had of showing it was through Catanatron's CLI, and for a rev1 demo presented to an audience less familar with RL, it would likely appear little progress had been made since rev0. This was a very valid point, and we decided to pivot our design once again, this time towards the development of a front-end interface that would allow users to play against our bot in a more engaging way, as well as better demonstrate the progress we had made with the bot itself. This was a significant change, as it meant we would be developing a full front-end interface, which was a significant undertaking, but we felt it was necessary in order to properly showcase our work. With only a couple weeks left until the rev1 demo, out first take on this UI was quite basic, but it was a good starting point and went over quite well in the demo.

Now having a working UI, we realised this was the ideal situation for usability testing, so we ran a round of interviews and surveys to gather feedback on the UI and overall user experience. This exposed a few key areas for improvement in the design. Similar to Prof. Smith in the demo, the users found the action log colours in the UI to be hard on the eyes, especially the red and blue text on a black background. They also disliked the look of the board, as some tile colours (such as lumber vs. wool) were hard to differentiate, and the overall look was quite bland. The action log colours were chosen to reflect the colour of each player, so we didn't remove them entirely, but we played around with different hue and saturation levels to find a more visually appealing colour scheme. We also redesigned the entire board, changing tile colours and textures. Users also commented on other issues, such as the lack of a clear indication of which bot

was which, as well as what the current dice roll was, so we added a bot name label and a dice roll display to the UI. Other changes were also made, you can see the details in the usability testing document in the extras folder. Follow-up usability testing showed a noticeable increase in the UX metrics we were tracking, indicating our changes were valuable.

While these changes were being made, we had continued meetings with Istvan. In one of these meetings, he expressed that having the sole goal of creating the best bot was a bit narrow, since likely someone will likely come along in the future with more compute resources and time and create something better. This was a good point, so we decided to take an idea given by Prof. Smith in the rev1 demo, and additionally utilize our bot as a learning tool. We accomplished this by adding an "explain move" feature to the UI, where users can receive LLM-generated explanations of why a bot made the move it did, and allowing users to walk back through a replay of the game. This allowed users to develop strategic understanding from the bots that they could then apply in their own games, supporting experiential learning.

4 Design Decisions (LO12)

[Reflect and justify your design decisions. How did limitations, assumptions, and constraints influence your decisions? Discuss each of these separately. —TPLT]

Our final design was heavily shaped by as much by limitations, assumptions, and constraints. At the beginning of the project, we imagined a system that would read a physical *Catan* board through computer vision and act as a real-time strategic coach. As the project progressed, we made a series of decisions that moved the system toward a web-based arena and analysis platform built around a trained bot, replay tooling, and explanation features. These decisions were responses to the realities of time, compute power, integration complexity, and demonstration value.

4.1 Limitations

The largest limitation was that our team had limited time to simultaneously build a strong reinforcement learning bot, a polished user interface, a full physical-board computer vision pipeline, and the surrounding infrastructure needed to make all of that reliable. Trying to fully realise every part of the original concept would likely have produced a weaker overall result in every area. Because of this, we chose to build on top of Catanatron rather than implementing our own simulator, and we deferred the physical-board computer vision component in favour of a web-based UI. This let us focus effort on the parts that most directly demonstrated technical progress, including the bot itself, the training and explanation pipelines, and the user interface.

A second limitation was compute power. Training in a game as large and stochastic as *Catan* is expensive, and while the use of department GPUs was critical to the project's success, it still wasn't sufficient infrastructure for us to

justify chasing model improvement through brute force alone. This limitation pushed us toward efficiency-oriented ideas such as curriculum learning, self-play, and reward shaping, allowing us to get more out of our training runs.

A third limitation was explainability. A strong model is not automatically a helpful one. Even when the bot makes good moves, the reason for those moves is not obvious to the user. This is a recognised problem in the field of RL, and it is especially important in a project like ours, where the bot is intended to be a learning tool for users. This limitation motivated the addition of replay support and LLM-based move explanations, since these features transform the project from being only a competitive bot into something that can also support learning.

4.2 Assumptions

One major assumption behind our design was that Catanatron would provide a sufficiently correct and flexible rule engine for both training and gameplay. This assumption justified building our architecture around integration rather than simulator creation. It also influenced our module boundaries. Rather than spending project time coding the board game rules from scratch, we treated the simulator as the stable core and built the learning, interface, and analysis layers around it. At times this assumption was challenged by simulator bugs and limitations, but when we ran into such issues, we were able to work around them or even contribute fixes to the Catanatron codebase, as it's open source.

Similarly, we assumed that the existing bots in Catanatron were correctly implemented, using only legal moves and not accessing board info unavailable to human players, such as other players cards. This assumption allowed us to use these bots as training opponents without worrying about them cheating or behaving in unrealistic ways. We were able to use them as consistent benchmarks for our bot's performance, and they gave us a constant goal to work towards, particularly Catanatron's AlphaBeta bot, which was the world's former best.

4.3 Constraints

The course deliverables imposed a documentation and traceability constraint. Because our design had to remain justifiable across the SRS, MG/MIS, and VnV documents, despite our often changing design ideas, we aimed for a relatively modular architecture with clear responsibilities, making it easier to add new components later (as we have throughout the last few milestones). It made the system easier both to reason about and to document.

Another constraint was the need to demonstrate progress in a way that was engaging and understandable to an audience that may not be familiar with the technical details of RL. This influenced our decision to build a front-end interface, and to add features that supported learning and engagement, such as move explanations and replay support.

Finally, we could not assume that users would be accessing our UI from a standard desktop environment, so we had to design for a variety of screen sizes

and input methods. This constraint influenced our design decisions around the UI, as every change was also tested on mobile to ensure it was still usable.

5 Economic Considerations (LO23)

[Is there a market for your product? What would be involved in marketing your product? What is your estimate of the cost to produce a version that you could sell? What would you charge for your product? How many units would you have to sell to make money? If your product isn't something that would be sold, like an open source project, how would you go about attracting users? How many potential users currently exist? —TPLT]

Our repo is open source, and we have no plans to sell anything produced during this project. Part of our motivation for this project was to push the field of *Catan* bots forward, and charging for access to our bot/arena would go against this motivation. As such, we'll focus instead in this section on how we will go about attracting users to our bot, and how many potential users currently exist.

Multiple members of our team are active in the *Catan* community, and we've made connections with other devs working on *Catan* bots throughout the project, thanks to are discussions in the Catanatron discord server. This has given us a good network of potential dev users for our system, and we plan to invite them to hook their own bots up to our bot arena, allowing them to benchmark their bots against ours. We are already talking to a few devs with promising bots, and hope to have their bots integrated soon. This will be a good way to attract users who are already interested in *Catan* bots, and we hope it will give our platform a reputation as a hub for good *Catan* bots, attracting more devs.

Of course, our focus is not only on dev users, but with more discussion about our tool, we believe we can attract a good number of regular *Catan* players as well, who are interested in playing against a strong bot or using the bot as a learning tool to improve their own play. We've already had some players from Colonist.io, a popular online *Catan* platform, express interest in testing themselves against our bot, and we plan to reach out to more players in the community through social media and *Catan* forums. Should a bot hosted on our platform ever surpass the strength of the best human players, we believe this would generate a lot of buzz and attract a large number of users to our platform, and this may not be all that far-fetched of a possibility given the progress in the field of *Catan* bots in recent years.

Catan is the most popular modern board game in the world, and Colonist.io alone has 4.3 million registered users, so while many may not be interested in testing themselves against or learning from our bot, we still have a large potential user base.

6 Reflection on Project Management (LO24)

[This question focuses on processes and tools used for project management. —TPLT]

6.1 How Does Your Project Management Compare to Your Development Plan

[Did you follow your Development plan, with respect to the team meeting plan, team communication plan, team member roles and workflow plan. Did you use the technology you planned on using? —TPLT]

Overall, our project management was fairly consistent with the development plan we proposed. We kept the core structure of weekly meetings, Discord-based communication, GitHub for source control, and issue tracking through GitHub Issues. Besides deferred computer vision-related tools, we also used all our expected technologies, including Python for development (holding to PEP 8 standard), React for the front-end, and GitHub Actions for CI/CD. That said, there are some differences from the original dev plan.

Our meeting time of 4:30–6:30 pm wasn't something we held to after the first couple of weeks. Instead, we found it was more effective to simply call for a meeting the night before whenever we had something important to discuss, as development on the project was rarely linear due to other course work and commitments. We would work less around midterms, and more in quiet periods, so a rigid schedule didn't make as much sense. That said, we still had regular meetings with our supervisor every Monday at 5:15 pm. Our meetings were also almost always virtual, as it was rare for us all to be on campus simultaneously. Our Discord server became quite the communication hub, having many channels for different purposes, and our supervisor was also added to the server, allowing for easy communication with him as well.

We also barely followed the proposed team member roles for the most part. Jake held the most consistent to his role as Team Lead, managing administrative tasks such as scheduling meetings and tasks, and acting as the main point of contact with our supervisor and TA, but Sunny, Rebecca, and Matthew all had fairly fluid roles, contributing to all aspects of the project as needed. This inconsistency likely results from poor initial role definitions. We should have focused more on technical roles rather than administrative ones, as in a 4-person team, most administrative tasks can be handled by a single person. We did split into natural technical roles however, with Jake and Sunny focusing more on backend development and training, while Rebecca and Matthew focusing more on the UI and documentation, but there was still some overlap.

6.2 What Went Well?

[What went well for your project management in terms of processes and technology? —TPLT]

A major thing that went well was communication. Discord was effective for day-to-day discussion, communicating with our supervisor, quick troubleshooting, and sharing resources, while GitHub Issues gave us a more durable place to record concrete tasks and feedback. The combination of both together worked well, and all team members were active on both platforms, allowing for good communication and collaboration.

Another strength was that we became better at dividing work by subsystem. As the project grew, it became more practical to let different people focus on RL, frontend/backend integration, documentation, testing, or research-heavy tasks while still checking in regularly. This reduced duplicated effort and let people build momentum in the parts of the project they were already touching.

We believe we also did an excellent job at flexibly adapting to new information and feedback, whether from our supervisor, TA, or users. We were able to quickly pivot our goals to adopt new ideas, and this led to a much stronger final product than if we had rigidly stuck to our original plans and been bogged down with computer vision troubles for example.

Finally, our tech choices were generally good, and we were able to use them effectively. We were all quite familiar with Python, so there were no struggles there, and React was a good choice for the front-end, making it easier to build an engaging UI than we would have been able to with a more basic framework. Our CI/CD pipeline was also quite effective, allowing us to catch bugs early and maintain a high level of code quality.

6.3 What Went Wrong?

[What went wrong in terms of processes and technology? —TPLT]

The biggest project-management issue was scope uncertainty early on. Because reinforcement learning work is hard to estimate, especially in a game as complex as *Catan*, it was difficult to predict how long training improvements, integrations, and tuning would actually take. That made long-range planning more fragile than it would be in a more conventional project. This was exacerbated by the fact that prior to this project, none of us had any experience with RL.

We also underestimated how much work would be involved in making the project presentable, not just functional. By rev0, we had meaningful technical progress, but not a strong enough presentation layer to communicate that progress clearly. That meant we had to make a fairly significant late-stage push into frontend and usability work, which although it worked out, was quite stressful at the time.

Finally, we had some struggles with documentation and traceability. Because our design was evolving so much, it was difficult to keep all the documentation up to date and ensure traceability across documents. This was especially true for the SRS, which we had to update multiple times as our design evolved.

6.4 What Would you Do Differently Next Time?

[What will you do differently for your next project? —TPLT]

The main thing we would do differently is establish a user-facing component earlier. Even if the underlying model was weak at first, having a basic end-to-end product sooner would have made early demos, and design decisions easier. It's much easier to iterate on a visible product than on a backend that can only show outputs through the command line.

We would also narrow the initial scope sooner and more aggressively. In hindsight, the original vision was quite ambitious, and we should have focused on a smaller set of core features that would have been more feasible to implement and demonstrate within the time constraints of the project. Making the simulator-based focus explicit earlier would have reduced some wasted planning effort around features that were ultimately deferred, and dealing with old docs now wouldn't be such a headache.

7 Ethics and Equity (LO18)

[Describe how your team applied the principle of equity and universal design to ensure equitable treatment of all stakeholders. —TPLT]

Our team tried to apply ethics and equity in two main ways, by thinking carefully about who the system is designed for and by avoiding design choices that would unfairly exclude or mislead users.

First, the project was designed to support users across a range of skill levels rather than assuming a single all users would be familiar or competent in *Catan*. A novice player and a competitive player do not need the same thing from a *Catan* system. By supporting multiple bot strengths, replay functionality, and explanatory features, we aimed to make the project useful to people with different backgrounds and goals. This is an equity consideration because it avoids privileging only already-expert users.

Second, we tried to make the interface more universally understandable. Usability testing showed that visual clarity cannot be taken for granted, especially when users are processing a dense game state. In response, we improved board readability, revised harsh colour choices, and added explicit labels, textures, and indicators rather than relying only on colour to convey information. This does not make the system perfectly accessible, and we believe there are still many UI improvements that could be made, but we attempted to move toward as clear and inclusive an interaction design as possible given time constraints.

Finally, making the repository open source is itself part of our ethical stance. We are not trying to build a gated system available only to people with money or institutional access. Instead, we want the project to be inspectable, usable, and extendable by other developers and interested players. Devs were also one of our stakeholder groups, and we wanted to ensure they had access to the code and the ability to build on it. Had Catanatron been closed-source, we would have had to build our own simulator, which would have entailed a long process

that may have wrought the entire project infeasible, so we choose to follow their example. This does not solve every equity issue, obviously, because technology still assumes access to hardware, internet, and time, but it does move the project in a more open and fair direction.

8 Reflection on Capstone

[This question focuses on what you learned during the course of the capstone project. —TPLT]

8.1 Which Courses Were Relevant

[Which of the courses you have taken were relevant for the capstone project? —TPLT]

Several courses were directly relevant to this capstone, with arguably 2AA4 the most relevant. That course emphasized iterative development, modular design, teamwork, and managing a larger project over time. Those lessons transferred almost directly into capstone, especially once our system grew into a multi-part architecture with backend, frontend, simulator integration, persistence, and testing concerns. 2AA4 was essentially a mini single-term capstone, only with requirements directly given.

Speaking of requirements, 3RA3 was another critical course, as it gave us a strong background in structured software documents, traceability, and design rationale. That background mattered a lot, especially when documents such as our SRS began diverging from the original template and had to stay consistent with each other. In fact, the Meyer template used in the SRS for this project was the same template taught in 3RA3s main project.

Machine learning coursework, especially 4AL3, was relevant to the technical core of the project. Even though the RL elements of the project quickly went beyond what that single course covers, having prior exposure to learning algorithms, model evaluation, and experimentation gave us a base to build on. Without that foundation, the technical details of the project would have been much harder to approach.

4HC3 was another important course, as it gave us prior experience with usability testing, user research, and design iteration. That experience was critical to our ability to gather and incorporate user feedback effectively, and it also gave us a good understanding of the importance of user-centered design, which was a key part of our approach to the project.

More generally, earlier programming and systems courses were also relevant. Even when a course did not map one-to-one onto the project, it often contributed something important, such as version control experience, or the ability to reason about performance and correctness.

8.2 Knowledge/Skills Outside of Courses

[What skills/knowledge did you need to acquire for your capstone project that was outside of the courses you took? —TPLT]

The capstone also required us to learn a large amount that was not cleanly covered by our coursework. The biggest area was practical deep reinforcement learning engineering. Early in the project, we each asynchronously worked through the Hugging Face Deep RL course, which was a great theoretical introduction to the field and the techniques we would be using. This is where we first learned about PPO, reward shaping, and self-play, which were all critical to the project.

It is one thing to understand the idea of reinforcement learning in theory, and another thing entirely to train, benchmark, debug, and improve a PPO-based agent in a large stochastic game. We had to implement and debug self-play, curriculum learning, benchmarking, model versioning, and experimental iteration from scratch, which proved quite the technical challenge.

We also had to learn the internals of the Catanatron ecosystem and how to adapt an existing open-source codebase to our own architecture. That meant learning someone else's abstractions, identifying extension points, and working around limitations in a codebase we did not originally design.

Frontend and full-stack integration were another major area of out-of-course learning. Building a user-facing interface around the bot required practical knowledge of React, TypeScript, Vite, Flask APIs, Dockerized development, and cross-layer debugging between a backend, frontend, and simulator. This was especially important because the final version of the project became much more interface-driven than originally planned.

Finally, we had to learn how to make AI outputs more interpretable. The explanation feature forced us to think about prompt design, what information should or should not be surfaced to users, and how to present strategic reasoning in a way that is helpful rather than cluttered. That kind of translation layer between model behaviour and user understanding was one of the most interesting parts of the project, and it sits well outside the boundaries of most standard software courses.